

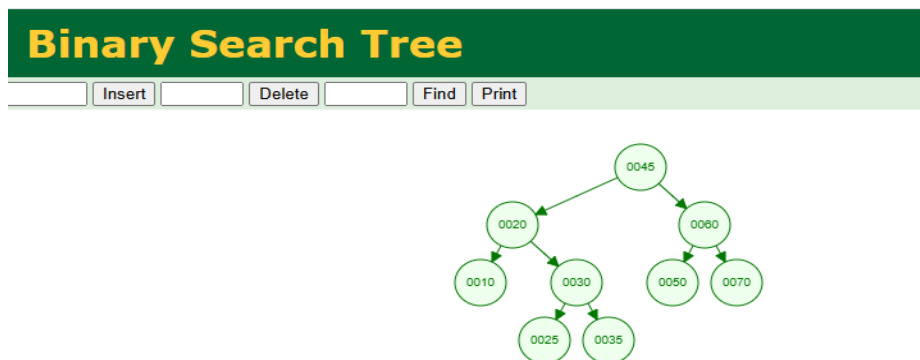
CO3- AT2- Tree Traversal Visualization

Visualize the Tree Traversal

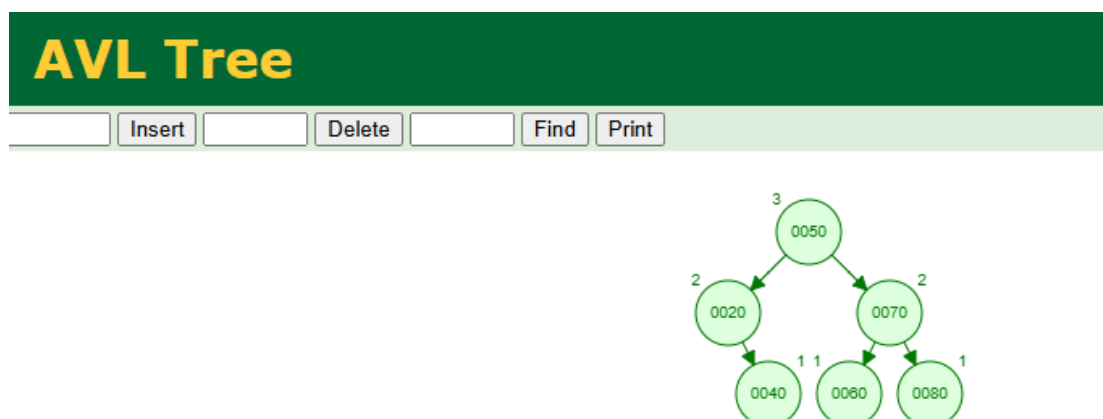
CSA0315 – Data Structures

S Maruthu Pandi- 192511162

1) Construct a Binary Search Tree (BST) using the dataset 45, 20, 60, 10, 30, 50, 70, 25, 35



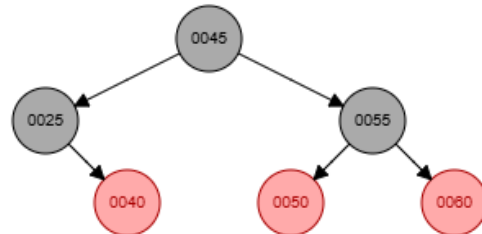
2) Construct an AVL Tree using the elements 50, 30, 70, 20, 40, 60, 80, perform the required operation and document the results. Delete node 30



3) Construct a Red-Black Tree using {45, 35, 55, 25, 40, 50, 60}, delete the node 35, and explain how violations are fixed using rotations and recoloring

Red/Black Tree

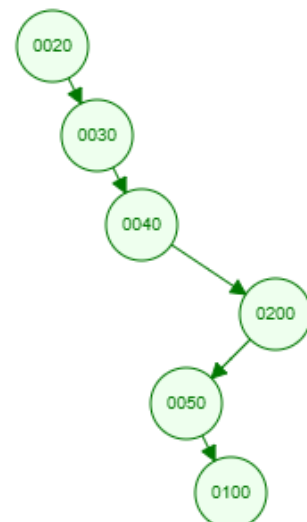
☐ Show Null Leaves



4) Develop a Splay Tree with elements {100, 50, 200, 40, 30, 20}, perform a search operation for 20, and demonstrate the zig-zig rotations that bring it to the root

Splay Tree

Element 0020 found.



5) Create a 2-3-4 Tree for the dataset {50, 40, 60, 30, 70, 20, 80}, perform insertions step-by-step, and show how 4-nodes are split during the process.

B-Trees

☐ Max.
Degree = 3
 ☐ Preemptive Split / Merge (Even max degree only)

☒ Max.
Degree = 4

☐ Max.
Degree = 5

☐ Max.
Degree = 6

☐ Max.
Degree = 7

```

graph TD
    Root["0040 | 0060"]
    L1["0020 | 0030"]
    L2["0050"]
    L3["0070 | 0080"]
    Root --> L1
    Root --> L2
    Root --> L3
  
```

6) Construct a Trie using the words {cat, car, care, dog, dot}, perform insertion for all words, search for the prefix “ca”, and delete the word “car” while maintaining the Trie structure.

Trie (Prefix Tree)

```

graph TD
    Root(( )) -- C --> C((C))
    Root -- D --> D((D))
    C -- A --> A((A))
    D -- O --> O((O))
    A -- R --> R((R))
    A -- T --> T1((T))
    R -- E --> E((E))
    O -- G --> G((G))
    O -- T --> T2((T))
    G -- T --> T3((T))
  
```